

CLASSI ASTRATTE E INTERFACCE

PIETRO DI LENA

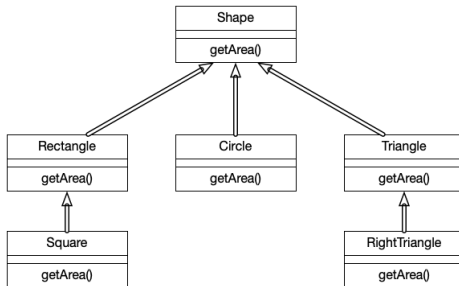
DIPARTIMENTO DI INFORMATICA – SCIENZA E INGEGNERIA
UNIVERSITÀ DI BOLOGNA

ALGORITMI E STRUTTURE DI DATI
ANNO ACCADEMICO 2024/2025



CLASSI ASTRATTE

- Una **classe astratta** è una superclasse generale che definisce caratteristiche comuni a tutta la gerarchia delle proprie sottoclassi
- Esempio: *getArea()* metodo comune nella gerarchia *figure geometriche*



- L'implementazione di *getArea()* è specifico per ogni figura geometrica
- La classe astratta *Shape* "impone" ad ogni sua sottoclasse di fornire una propria implementazione del metodo *getArea()*

CLASSI ASTRATTE IN JAVA

- In Java dichiariamo classi/metodi astratti con la keyword **abstract**

```
1 public abstract class Shape { // Classe astratta
2     public abstract double getArea(); // Metodo astratto
3 }
```

- Un metodo astratto è solo dichiarato ma non ha implementazione
- Una classe **deve** essere dichiarata astratta se ha un metodo astratto
- L'implementazione del metodo astratto è lasciata alle sottoclassi

```
1 public class Rectangle extends Shape {
2     private double width, length;
3     public double getArea() { // Implementazione del metodo
4         return width*length;
5     }
6 }
```

- Una classe astratta è un tipo ma non può essere istanziata

```
1 Shape    f = new Shape();      // Errore
2 Rectangle r = new Rectangle(); // Corretto
3 Shape    x = new Rectangle(); // Corretto
```

ULTERIORI PRECISAZIONI SULLE CLASSI ASTRATTE

- Una classe astratta può
 - avere variabili locali (ereditate dalle sottoclassi)
 - esporre metodi implementati (anche questi ereditati)
 - avere un costruttore (invocabile dalle sottoclassi)

```
1 import java.awt.Color;
2 public abstract class Shape { // Classe astratta
3     private Color color;
4
5     public Shape(Color color) {
6         this.color = color
7     }
8     public Shape() {
9         this.color = Color.BLACK;
10    }
11    public void setColor(Color color) { // Metodo pubblico
12        this.color = color;
13    }
14    public abstract double getArea(); // Metodo astratto
15 }
```

A COSA SERVONO LE CLASSI ASTRATTE?

- Ci permettono di strutturare logicamente il codice
- Definire metodi astratti comuni a tutte le sottoclassi
 - Caratteristiche comuni con implementazione specifica
- Definire metodi di default (implementati) per tutte le sottoclassi
 - Caratteristiche comuni con implementazione comune
- In generale, fornire una struttura di base per tutte le sottoclassi

INTERFACCE

- Un'interfaccia è un modello che funge da schema per una classe
- Modello di classe ancora più ridotto della classe astratta
 - Non contiene nessuna implementazione
 - Contiene solo metodi pubblici e astratti
 - Non specifica costruttori
 - Può contenere solo variabili costanti, statiche e pubbliche
- Si definisce con la keyword `interface` (al posto di `class`)
- Esempio:

```
1 public interface Shape {  
2     public static final double PI = 3.14;  
3     public double getArea();  
4     public double getPerimeter();  
5 }
```

Nota: non è necessario indicare esplicitamente i metodi come *abstract*

IMPLEMENTARE UN'INTERFACCIA

- Usiamo **implements** per definire una classe che implementa un'interfaccia
- Tutti i metodi specificati nell'interfaccia devono essere implementati

```
1 public class Circle implements Shape {  
2     private final double radius;  
3  
4     public Circle(double radius) {  
5         this.radius = radius;  
6     }  
7  
8     public double getArea() {  
9         return Shape.PI*radius*radius;  
10    }  
11  
12    public double getPerimeter() {  
13        return 2*Shape.PI*radius;  
14    }  
15 }
```

INTERFACCE MULTIPLE

- In Java una classe può implementare più interfacce

```
1 import java.awt.Color;
2 public interface Colorable {
3     public void setColor(Color c);
4     public Color getColor();
5 }
```

```
1 import java.awt.Color;
2 public class Circle implements Shape, Colorable {
3     private final double radius;
4     private Color color;
5     public Circle(double radius) {
6         this.radius = radius;
7         this.color = Color.BLACK;
8     }
9     public double getArea() {return Shape.PI*radius*radius;}
10    public double getPerimeter() {return 2*Shape.PI*radius;}
11    public void setColor(Color c) {color = c;}
12    public Color getColor() {return color;}
13 }
```

La classe *Circle* è sia di tipo *Shape* che *Colorable*

CLASSI ASTRATTE VS INTERFACCE

Proprietà	Interfaccia	Classe Astratta
Ereditarietà	Una classe può implementare più interfacce	Una classe può estendere una sola classe astratta
Implementazione	Non fornisce implementazioni	Può fornire implementazioni
Variabili	Solo se <i>static</i> , <i>final</i> e <i>public</i>	Può avere variabili di ogni tipo
Modificatori	Definisce solo metodi pubblici	Può definire metodi con diversa visibilità
Tipo	E' un tipo di dato	E' un tipo di dato
Istanziabilità	Non può essere istanziata	Non può essere istanziata

INTERFACCIA COMPARABLE

- La libreria Java fornisce diverse interfacce predefinite
- Un'interfaccia molto utilizzata è [Comparable](#)
- Due oggetti di tipo *Comparable* sono confrontabili rispetto a $<$, $>$, $=$
- L'interfaccia *Comparable* definisce un solo metodo

```
public int compareTo(Object o);
```

Nota: l'istestazione vera è leggermente differente (capiremo perché)

- Le classi che implementano tale metodo devono ritornare
 - **int negativo**: se l'oggetto su cui il metodo viene chiamato **precede** (in qualche ordinamento) l'oggetto *o* passato come parametro
 - **zero**: se l'oggetto è **uguale** (risp. all'ordinamento) al parametro
 - **int positivo**: se l'oggetto **segue** (nell'ordinamento) il parametro

ESEMPIO DI CLASSE COMPARABLE

```
1 import java.awt.Color;
2 public class Circle implements Shape,Colorable,Comparable {
3     private final double radius;
4     private Color color;
5     public Circle(double radius) {
6         this.radius = radius;
7         this.color = Color.BLACK;
8     }
9     public double getArea() {return Shape.PI*radius*radius;}
10    public double getPerimeter() {return 2*Shape.PI*radius;}
11    public double getRadius() {return radius;}
12    public void setColor(Color c) {color = c;}
13    public Color getColor() {return color;}
14
15    public int compareTo(Object o) {
16        if(o != null && o instanceof Circle) {
17            Circle c = (Circle)o; // typecasting necessario
18            return (int)(this.radius - c.getRadius());
19        }
20        return -1; // Default se o non è di tipo Circle o null
21    }
22 }
```

Gli oggetti *Circle* sono adesso confrontabili e ordinabili rispetto al raggio

- Implementare l'interfaccia della struttura dati *Dictionary* in Java
 - Primo tentativo, miglioreremo l'interfaccia successivamente
- Commentare i metodi ed il loro funzionamento
- Ricordiamo che la struttura dati Dizionario espone i metodi
 - SEARCH(Key k): cerca e ritorna l'oggetto associato alla chiave k
 - INSERT(Key k, Data d): inserisce la coppia (k,d) nel dizionario
 - DELETE(Key k): rimuove i dati associati alla chiave k
- Attenzione: come dovremmo definire il tipo della chiave e dei dati?
 - Vogliamo un'interfaccia che ci permetta di implementare dizionari per qualsiasi tipo di chiave e dati